

Pointwise-Optimal Computational Complexity: Sorting under Partial Information against Hidden Completion

Yunbei Xu
National University of Singapore
yunbei@nus.edu.sg

Abstract

Worst-case complexity can erase the structure of the public instance. Sorting under partial information gives a finite model where the pointwise refinement becomes exact. Given a public partial order P , an algorithm must identify an unknown linear extension using additional comparisons. If $e(P)$ counts the compatible extensions, then the optimal worst-case number of additional comparisons after P is revealed is $\Theta(\log_2 e(P))$, and one uniform algorithm attains this bound for every P . Thus, when the information revealed to the algorithm is treated as the instance, universal instance-optimality is possible against the worst-case hidden completion: the global algorithm matches a clairvoyant benchmark that knows P but not the hidden linear extension. The no-information public order recovers the usual $\Theta(n \log n)$ bound; a chain has zero additional comparison cost. Finally, $\log e(P)$ has Occam and polyhedral interpretations, including Stanley's formula $\text{Vol}(\mathcal{O}(P)) = e(P)/n!$. The note uses this solved example to clarify the quantifiers of pointwise computational complexity and to relate them to pointwise statistical dimensions and information-relaxation viewpoints.

1 Introduction

Worst-case analysis asks for one number. For comparison sorting on n items, the answer is $\Theta(n \log n)$. The answer is correct, but it is too compressed: it gives the same label to an entirely unordered list and to a list whose order is almost already known.

Sorting with partial information is the cleanest place where this compression can be undone. The algorithm is given a partial order P on the items. The hidden truth is an arbitrary total order extending P . The algorithm may ask additional comparisons and must recover the hidden total order. If

$$e(P) := |\text{Lin}(P)|$$

denotes the number of linear extensions of P , then $\log_2 e(P)$ is the number of remaining binary decisions. The theorem says that this is not just an information lower bound. It is the correct pointwise computational complexity, up to a universal constant.

This note is therefore intentionally modest in its technical claims and precise in its interpretation. It does not introduce a new sorting algorithm. It reformulates the classical and modern theory of sorting under partial information as a solved benchmark for pointwise worst-case complexity (Fredman, 1976; Kahn and Kim, 1995; Cardinal et al., 2013; Haeupler et al., 2025; Rutschmann, 2026). The public point is P ; the hidden states are $\text{Lin}(P)$; the resource is the number of additional comparisons; and the pointwise dimension is

$$d_{\text{sort}}(P) = \log_2 |\text{Lin}(P)|.$$

The broader lesson is about information revelation. If the revealed information is formulated as the public instance, then one can ask for a profile that is pointwise worst-case over the remaining hidden completions. The benchmark may be clairvoyant about the public instance P : it may choose the best decision tree after seeing P , but it still must work for every hidden extension in $\text{Lin}(P)$. The theorem below shows universal instance-optimality in this sense. A single globally valid algorithm matches, for every P , the same order of comparisons as this instance-wise clairvoyant benchmark.

The important quantifier is that the algorithm is still global. Pointwise complexity does not mean choosing a different algorithm after seeing the hidden order. Rather, one fixed algorithm must be correct for every partial order and every compatible extension, while its cost is evaluated as a function of the public structure P . In this sense the result is a genuine refinement of worst-case sorting: the no-information public order, often called the antichain, has no known comparisons and leaves all $n!$ orders possible, so it gives $\log n! = \Theta(n \log n)$; a chain is a completely known order and gives zero remaining comparisons; and every intermediate partial order receives its own exact scale.

There is also a useful conceptual warning. In this finite comparison model, $\log e(P)$ is close to an Occam length: after conditioning on P , there are exactly $e(P)$ possible hidden objects. This is the finite endpoint of a broader localization principle, not the whole principle. In uncountable statistical models, singletons typically have prior mass zero, and pointwise statistical dimensions must use finite-resolution metric balls around the realized hypothesis. Section 7 makes the mapping precise, and also records the closely related polyhedral interpretation through linear inequalities and order polytopes.

2 Pointwise profiles

We use only the minimal language needed for the sorting example. Fix a computational model, a correctness requirement, and a resource measure, in the spirit of Blum’s axiomatic treatment of complexity (Blum, 1967). At input size n , let $x \in \mathcal{X}_n$ denote the public structure. There may also be hidden completions $\omega \in \Omega(x)$ consistent with x . A globally valid algorithm is one algorithm that is correct for every public structure and every compatible hidden completion. Its pointwise worst-case resource profile is

$$p_{A,n}(x) := \sup_{\omega \in \Omega(x)} \text{Time}(A, x, \omega), \quad x \in \mathcal{X}_n.$$

When there is no hidden completion, $\Omega(x)$ is a singleton and this reduces to the ordinary running-time profile.

Worst-case complexity collapses the profile to $\sup_x p_{A,n}(x)$ and then optimizes over algorithms. Pointwise complexity keeps the function $x \mapsto p_{A,n}(x)$, but it keeps the same uniformity discipline: the algorithm is fixed before the input is chosen. A representation of why a point is easy—a tree decomposition, active set, preconditioner, entropy vector, decision tree, or heap trace—may be crucial in a proof or implementation. It is not a primitive part of the pointwise complexity unless the computational model makes it part of the input, and if an algorithm constructs it, its construction cost must be charged.

The strongest form of a pointwise theorem compares one global algorithm with the conditional optimum at each public point. For a public structure x , let $C^\star(x)$ be the least worst-case resource needed after x is fixed, allowing the decision procedure to depend on x . This is a clairvoyant benchmark only with respect to the revealed instance: it knows x , but it does not know which hidden completion $\omega \in \Omega(x)$ will occur. A single algorithm A is pointwise optimal up to a constant if

$$p_{A,n}(x) \leq C C^\star(x) \quad \text{for every } x \in \mathcal{X}_n$$

with a universal constant C . We will call this phenomenon universal instance-optimality of the pointwise profile. This is the quantifier pattern used below: first prove a conditional lower bound at every P , then show that one uniform sorting algorithm matches all of those lower bounds simultaneously.

3 Sorting under partial information

Let $[n] = \{1, \dots, n\}$. A partial order P on $[n]$ is public information. A linear extension of P is a permutation π of $[n]$ such that whenever $i <_P j$, item i appears before item j in π . We write $\text{Lin}(P)$ for the set of linear extensions and $e(P) = |\text{Lin}(P)|$.

The hidden input is an unknown $\pi \in \text{Lin}(P)$. An algorithm receives P , may adaptively query pairwise comparisons, and must output π . A query on (i, j) returns which of i and j appears first in the hidden extension. Comparisons already forced by P are public and need not be queried; the comparison cost counts only additional queries to the hidden linear order.

For a deterministic algorithm A , let $q_A(P, \pi)$ be the number of additional comparisons made on hidden extension π , and define the public-point worst-case profile

$$Q_A(P) := \max_{\pi \in \text{Lin}(P)} q_A(P, \pi).$$

The conditional optimum at P is

$$C^*(P) := \inf_A Q_A(P),$$

where the infimum is over deterministic comparison decision procedures that are correct for every $\pi \in \text{Lin}(P)$. Equivalently, $C^*(P)$ is the minimum height of a comparison decision tree, allowed to be designed after P is fixed, that identifies every linear extension of P . The upper bound in Theorem 4.1 is stronger: it is achieved by a single uniform algorithm that receives P as input.

Definition 3.1 (Pointwise sorting dimension). *The pointwise comparison dimension of a partial order P is*

$$d_{\text{sort}}(P) := \log_2 e(P).$$

It is the number of binary comparison bits, in Shannon's logarithmic sense, that remain after the public information P has been revealed (Shannon, 1948).

4 The main theorem

Theorem 4.1 (Pointwise optimality of partial-information sorting). *For every finite partial order P ,*

$$C^*(P) \geq \lceil \log_2 e(P) \rceil.$$

Moreover, there exists a single deterministic algorithm A_{opt} , valid for every finite partial order, such that

$$Q_{A_{\text{opt}}}(P) = O(\log e(P))$$

for every P , with the convention that $Q_{A_{\text{opt}}}(P) = 0$ when $e(P) = 1$. Hence

$$C^*(P) = \Theta(\log e(P))$$

for all P with $e(P) \geq 2$, and $C^(P) = 0$ exactly when $e(P) = 1$.*

If the public information is given as an acyclic directed graph $G = (V, E)$ whose reachability order is P_G , then known implementations sort using $O(\log e(P_G))$ additional hidden-order comparisons. Total running time depends on the input and preprocessing convention. Under the standard DAG-input convention, topological heapsort runs in $O(|V| + |E| + \log e(P_G))$ time and $O(\log e(P_G))$ hidden-order comparisons (Haeupler et al., 2025). In the preprocessing formulation, the unified-bound-heap algorithm gives $O(|E|)$ preprocessing and then $O(\log e(P_G))$ sorting time and comparisons in that model (Rutschmann, 2026).

Proof. The lower bound is the decision-tree counting bound, applied after conditioning on P . Fix P . A deterministic comparison procedure induces a binary decision tree on the answers that are not already forced by P ; branches inconsistent with P may be deleted. Each root-to-leaf path is a sequence of additional comparison answers. At a leaf, the procedure outputs one linear extension of P . If two distinct extensions $\pi, \pi' \in \text{Lin}(P)$ reached the same leaf, then the procedure would output the same order on both hidden inputs and therefore would be wrong on at least one of them. Thus the tree must have at least $e(P)$ leaves. A binary tree of height h has at most 2^h leaves, so $h \geq \lceil \log_2 e(P) \rceil$. Taking the minimum over all decision trees for this fixed P gives $C^*(P) \geq \lceil \log_2 e(P) \rceil$.

For the upper bound we invoke the classical theorem on sorting under partial information. Kahn and Saks proved the balancing principle for poset extensions, Kahn and Kim gave a polynomial-time entropy-based sorting algorithm using $O(\log e(P))$ comparisons, and later work gave simpler and faster implementations, culminating in the near-linear and preprocessing-optimal algorithms cited in the statement (Kahn and Saks, 1984; Kahn and Kim, 1995; Cardinal et al., 2013; van der Hoog and Rutschmann, 2024; van der Hoog et al., 2025; Haeupler et al., 2025; Rutschmann, 2026). These algorithms are uniform: the same algorithm works for every input partial order or DAG. Applying such an algorithm to P gives $Q_{A_{\text{opt}}}(P) = O(\log e(P))$. If $e(P) = 1$, the public order has a unique linear extension, so that extension can be output without querying the hidden order. Combining the lower and upper bounds proves the theorem. $\square$

The theorem is stronger than ordinary worst-case optimality. Taking the supremum over all partial orders on n items gives the usual comparison-sorting bound. Indeed, for every partial order P , the set $\text{Lin}(P)$ is a subset of all $n!$ permutations, so $e(P) \leq n!$; equality holds exactly in the no-information case, where no two distinct items are publicly comparable. Thus ordinary worst-case sorting is the maximum of the pointwise profile, while the same globally valid algorithm also pays only for the remaining extension entropy at every public point P .

Equivalently, Theorem 4.1 is an instance-optimality theorem with the instance chosen correctly. The instance is not the hidden total order π , because no correct algorithm may know π in advance. The instance is the revealed partial order P . Against the clairvoyant benchmark that knows P and designs the best comparison tree for $\text{Lin}(P)$, the uniform algorithm loses only a constant factor. This is the precise sense in which pointwise worst-case profiles are compatible with the usual complexity-theoretic discipline of comparing to an instance-wise optimum without allowing the actual solution to be revealed.

Corollary 4.2 (A pointwise refinement of worst-case sorting). *Let $\mathcal{P}_n$ be the family of partial orders on $[n]$. There is a globally valid deterministic comparison-sorting algorithm A_{opt} whose profile satisfies*

$$Q_{A_{\text{opt}}}(P) = \Theta(d_{\text{sort}}(P))$$

for every $P \in \mathcal{P}_n$ with $e(P) \geq 2$, while

$$\sup_{P \in \mathcal{P}_n} Q_{A_{\text{opt}}}(P) = \Theta(n \log n).$$

Thus $P \mapsto \log e(P)$ is an exact pointwise refinement of the scalar worst-case sorting complexity.

5 Examples

No information: the worst public point. An antichain is simply the partial order with no known comparison outcomes: no two distinct items are publicly comparable. Then every permutation is compatible with the public information, so $e(P) = n!$. Stirling's formula gives

$$d_{\text{sort}}(P) = \log_2 n! = \Theta(n \log n).$$

Since no partial order can have more than $n!$ linear extensions, this no-information case is the public point that recovers the ordinary worst-case theorem for comparison sorting.

Complete information. If P is a chain, then $e(P) = 1$. The hidden order is already determined. The pointwise cost is zero additional comparisons, even though the input size is still n . If total running time rather than comparisons is measured, the cost of reading and outputting the public order must of course be charged.

Ordered blocks. Suppose the items are partitioned into blocks $B_1, \dots, B_r$, the public information says that every item in B_a precedes every item in B_b whenever $a < b$, and there is no information inside a block. Then

$$e(P) = \prod_{a=1}^r |B_a|!, \quad d_{\text{sort}}(P) = \sum_{a=1}^r \log_2(|B_a|!).$$

The dimension adds across independent unresolved regions. A good algorithm should spend comparisons inside the blocks and none across blocks; the theorem says that, up to constants, a single global algorithm achieves exactly this behavior.

Merging two sorted lists. If P consists of two chains of lengths a and b , then sorting is merging. The number of compatible total orders is

$$e(P) = \binom{a+b}{a},$$

so

$$d_{\text{sort}}(P) = \log_2 \binom{a+b}{a} = (a+b)H_2\left(\frac{a}{a+b}\right) + O(\log(a+b)),$$

where H_2 is the binary entropy function. The information-theoretic cost of merging is therefore another point on the same profile.

6 Clarifying resources and instances in pointwise complexity

The partial-information sorting theorem is a clean model because every object is visible. The instance is the public point P . The remaining state space is $\text{Lin}(P)$. The resource is additional comparisons. The dimension is the logarithm of the remaining state space. The theorem proves that this dimension is both necessary and sufficient for computation in the comparison model.

This clarifies the role of representations. A DAG representation, a transitive closure, a transitive reduction, a graph-entropy computation, a balanced decision tree, or a heap trace can be very

important for proving or implementing the upper bound. But the comparison dimension $\log e(P)$ does not depend on which representation is used. If one changes the resource from comparisons to total running time, the representation immediately becomes part of the model: reading a sparse DAG, querying a partial-order oracle, and preprocessing a closure are different computational tasks.

The value $e(P)$ is a complexity profile, not necessarily a quantity the algorithm must compute exactly. Counting linear extensions is $\#P$ -complete (Brightwell and Winkler, 1991). Thus the significance of the upper bound is not that the algorithm first evaluates $e(P)$, but that its behavior can be analyzed by $\log e(P)$ without exact counting.

The theorem also separates pointwise analysis from neighboring notions. Average-case analysis integrates the profile over a distribution on public partial orders. Generic-case analysis asks for a bound on a large subset of public partial orders. Parameterized analysis upper-bounds the profile by width, the number of incomparable pairs, poset dimension, or another chosen parameter. These are useful summaries. They are not the profile itself. For sorting under partial information, the intrinsic profile is exactly $P \mapsto \log e(P)$, up to constants.

7 Connections with statistical, algorithmic, and polyhedral notions

The present theorem has three close relatives: Occam coding, polyhedral volume, and pointwise statistical dimension. They share a localization principle, but they do not have the same units. The safest way to relate them is to begin with the finite Occam specialization.

Let S be a finite nonempty set and let $\Delta(S)$ be the set of probability distributions on S . For $\mu \in \Delta(S)$, define the singleton pointwise code length

$$D_\mu(s) := \log_2 \frac{1}{\mu(s)}, \quad s \in S,$$

with the convention $D_\mu(s) = +\infty$ if $\mu(s) = 0$. Then

$$\inf_{\mu \in \Delta(S)} \sup_{s \in S} D_\mu(s) = \log_2 |S|.$$

Indeed, for every μ , some point has mass at most $1/|S|$, while the uniform distribution attains equality. Applying this identity to $S = \text{Lin}(P)$ gives

$$d_{\text{sort}}(P) = \inf_{\mu \in \Delta(\text{Lin}(P))} \sup_{\pi \in \text{Lin}(P)} \log_2 \frac{1}{\mu(\pi)}.$$

Thus $\log e(P)$ is exactly the optimal worst-case Occam length of the finite set of hidden total orders still compatible with P (Blumer et al., 1987).

The same number has a polyhedral, and hence linear-programming, shadow. The partial order P defines Stanley's order polytope

$$\mathcal{O}(P) := \{x \in [0, 1]^n : x_i \leq x_j \text{ whenever } i \leq_P j\}.$$

This is a feasible region cut out by linear inequalities. Its canonical triangulation has one simplex of volume $1/n!$ for each linear extension of P , and Stanley's formula gives

$$\text{Vol}(\mathcal{O}(P)) = \frac{e(P)}{n!} \quad \text{or equivalently} \quad \log e(P) = \log n! + \log \text{Vol}(\mathcal{O}(P)).$$

Thus $\log e(P)$ can also be read as the discrete log-volume of the remaining feasible ranking region (Stanley, 1986). This does not make partial-information sorting an algorithm for general linear

programming. The sorting problem identifies a hidden ranking by adaptive comparison queries, whereas linear programming optimizes a linear objective over a polytope. But the analogy is useful: revealed inequalities shrink a feasible region, and the pointwise complexity measures the remaining combinatorial or volumetric size of that region.

There is an equivalent vertex view through the linear-ordering polytope. Encode a total order by pairwise variables $y_{ij} \in \{0, 1\}$, where $y_{ij} = 1$ means that i precedes j . The public constraints $i <_P j$ fix the corresponding coordinates to one, and the remaining feasible ranking vertices are precisely the linear extensions of P . A comparison query asks for one still-unfixed coordinate of the hidden vertex. In this form, partial-information sorting is the problem of identifying a hidden vertex of a public face of a ranking polytope.

The relation to pointwise statistical dimension is obtained by replacing finite singletons by finite-resolution metric balls. If S is equipped with the discrete metric $\rho(s, t) = \mathbf{1}\{s \neq t\}$, then for every $\varepsilon < 1$, the ball $B_\rho(s, \varepsilon)$ is the singleton $\{s\}$, and the finite identity above becomes

$$\inf_{\mu \in \Delta(S)} \sup_{s \in S} \log_2 \frac{1}{\mu(B_\rho(s, \varepsilon))} = \log_2 |S|.$$

The pointwise statistical dimension framework starts from the same prior-mass template but removes the finite/discrete restriction (Li and Xu, 2026). There the hypothesis class $\mathcal{F}$ may be uncountable, the distance ρ is a statistical semimetric, and the pointwise quantity has the form

$$D_\Pi(f, \varepsilon) := \log \frac{1}{\Pi(B_\rho(f, \varepsilon))}, \quad B_\rho(f, \varepsilon) = \{g \in \mathcal{F} : \rho(f, g) \leq \varepsilon\},$$

possibly with optimization or integration over scales. The ball is essential: under a continuous prior, $\Pi(\{f\}) = 0$ for typical f , so the singleton Occam length would be infinite. Finite-scale neighborhoods turn the question into: how much prior mass lies in a statistically indistinguishable neighborhood of this hypothesis? In smooth or representation-rich models, that mass is governed by local geometry, effective rank, and learned feature spectra rather than by finite cardinality.

The mapping is therefore exact but typed:

finite sorting state space	$\text{Lin}(P)$,
finite Occam envelope	$\log \text{Lin}(P) $,
polyhedral shadow	$\log n! + \log \text{Vol}(\mathcal{O}(P))$,
discrete metric ball at $\varepsilon < 1$	$\{\pi\}$,
statistical analogue	$\log 1/\Pi(B_\rho(f, \varepsilon))$,
resource controlled here	additional comparisons,
resource controlled in learning	generalization or estimation error.

The shared principle is localization before taking the worst case. The mathematical objects and units are different. The present theorem is not a statistical generalization bound; it is a comparison-decision theorem. Conversely, the pointwise dimension of Li and Xu (2026); Lutz (2016) is not a sorting complexity; it is a finite-scale statistical complexity for hypothesis classes that may be uncountable.

The same distinction clarifies the relation with algorithmic information complexity (Xu, 2026b). A code length can be assigned to the realized extension π conditional on P , and some extensions may have very short descriptions. Sorting under partial information asks for a minimax guarantee: one fixed comparison algorithm must be correct for every $\pi \in \text{Lin}(P)$, and the profile takes the

worst case over this compatible set. The operational length is therefore the minimax code length of the remaining set, $\log |\text{Lin}(P)|$, not the description length of a lucky realized extension.

There is also a high-level connection with information-relaxation formulations of algorithmic robustness. For example, [Xu \(2026a\)](#) uses minimax language to compare what becomes possible when additional information is revealed to an algorithmic procedure. The present note studies the offline computational analogue in its cleanest finite form: reveal P , take the worst case over $\text{Lin}(P)$, and compare a global algorithm with the benchmark that is allowed to know P . This shared quantifier pattern should not be confused with a shared formal framework. Here the resource is additional comparisons; in statistical or robustness problems the resource may be risk, loss, sample size, or an algorithm-dependent performance gap.

Thus pointwise does not mean nonuniform, and localized does not always mean geometric. In finite comparison problems, localization may reduce to an Occam-like finite state count. In uncountable statistical problems, localization requires finite-resolution metric balls. In polyhedral problems, localization may appear as the volume or vertex structure of a feasible region. In all cases, the discipline is the same: fix the global rules first, and only then keep the pointwise profile instead of collapsing it immediately to its maximum.

8 Outlook

Sorting with partial information is a solved benchmark for pointwise worst-case complexity. The residual dimension $\log e(P)$ is explicit, the lower bound is elementary, and nontrivial algorithms attain the bound uniformly. More difficult problems have the same shape but are missing one of these ingredients. For shortest paths, one can count possible distance orderings of a fixed graph topology, but exact universal optimality depends on the data-structure model. For optimization, local curvature or error-bound constants often predict fast convergence, but the globally valid method and the cost of discovering favorable structure must be included. For learning, pointwise statistical dimensions can control risk, but their computational evaluation and the training-time profile are separate questions.

The moral is deliberately narrow and useful. Partial-information sorting is not an uncountable geometry theorem and not a general linear-programming theorem. It is the finite endpoint where the residual uncertainty is exactly countable, comparison decisions can realize the corresponding code length, and the quantifiers are completely visible. The public information is P , the hidden states are $\text{Lin}(P)$, the resource is additional comparisons, the lower bound is a decision tree, and the upper bound is one uniform algorithm.

This is the lesson the example exports. First decide what information is legitimately revealed, and make that revealed information the public instance. Then compare one global algorithm with the clairvoyant benchmark that knows this instance but not the hidden completion. When the global profile matches the benchmark point by point, worst-case complexity has a genuine pointwise refinement, and universal instance-optimality is achieved without abandoning uniform computation.

Acknowledgements

This note is primarily a reformulation of the modern theory of sorting under partial information. It identifies that theory as an exact example of pointwise worst-case computational complexity.

References

- Manuel Blum. A machine-independent theory of the complexity of recursive functions. *Journal of the ACM*, 14(2):322–336, 1967.
- Anselm Blumer, Andrzej Ehrenfeucht, David Haussler, and Manfred K. Warmuth. Occam’s razor. *Information Processing Letters*, 24(6):377–380, 1987.
- Graham Brightwell and Peter Winkler. Counting linear extensions. *Order*, 8:225–247, 1991.
- Jean Cardinal, Samuel Fiorini, Gwenael Joret, Raphael M. Jungers, and J. Ian Munro. Sorting under partial information (without the ellipsoid algorithm). *Combinatorica*, 33:655–697, 2013.
- Michael L. Fredman. How good is the information theory bound in sorting? *Theoretical Computer Science*, 1(4):355–361, 1976.
- Bernhard Haeupler, Richard Hladik, John Iacono, Vaclav Rozhon, Robert E. Tarjan, and Jakub Tetek. Fast and simple sorting using partial information. In *Proceedings of the 2025 Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pages 3953–3973. SIAM, 2025.
- Jeff Kahn and Jeong Han Kim. Entropy and sorting. *Journal of Computer and System Sciences*, 51(3):390–399, 1995.
- Jeff Kahn and Michael E. Saks. Balancing poset extensions. *Order*, 1:113–126, 1984.
- Shaojie Li and Yunbei Xu. Pointwise generalization in deep neural networks. arXiv:2605.18598, 2026.
- Neil Lutz. A note on pointwise dimensions. arXiv:1612.05849, 2016.
- Daniel Rutschmann. Sorting under partial information with optimal preprocessing time via unified bound heaps. arXiv:2604.12653, 2026.
- Claude E. Shannon. A mathematical theory of communication. *Bell System Technical Journal*, 27(3):379–423 and 27(4):623–656, 1948.
- Richard P. Stanley. Two poset polytopes. *Discrete & Computational Geometry*, 1:9–23, 1986.
- Ivor van der Hoog and Daniel Rutschmann. Tight bounds for sorting under partial information. In *Proceedings of the 65th IEEE Symposium on Foundations of Computer Science (FOCS)*, 2024.
- Ivor van der Hoog, Eva Rotenberg, and Daniel Rutschmann. Simpler optimal sorting from a directed acyclic graph. In *Proceedings of the 2025 Symposium on Simplicity in Algorithms (SOSA)*, 2025.
- Yunbei Xu. Pointwise complexity for Gaussian fields: Upper envelopes, algorithmic lower bounds, and separation. arXiv:2606.07931, 2026.
- Yunbei Xu. Bellman-sufficient Information Complexity. arXiv:2606.11171, 2026.